TP n°3 - DevOps
Tests Automatisés avec
GitHub Actions et Jenkins

Contexte et Objectifs

Vous travaillez dans une startup qui développe plusieurs microservices :

- Le premier est écrit en **C# (.NET)** et contient des tests avec **MSTest**.
- Le second est écrit en **Python**, avec des tests **unittest**.

L'entreprise souhaite **automatiser l'exécution des tests** à chaque commit, d'abord sur **GitHub Actions**, puis sur **Jenkins**.

Dans ce TP, vous allez :

- Exécuter automatiquement des tests unitaires dans un pipeline CI.
- Configurer un workflow GitHub Actions pour les tests .NET et Python.
- Créer un pipeline Jenkins équivalent.
- Comprendre la logique des déclencheurs, des jobs et des rapports de tests.

Partie 1 : Tests Automatisés avec GitHub Actions

A. Préparation du projet .NET

1. Sur votre disque dur, créez un dossier nommé « **TP3-DevOps** ».
2. Dans ce dernier, créez un sous-dossier **« dotnet-app** », puis créez dedans les deux fichiers suivants :

   - « **MathUtils.cs** » contenant le code donné ci-dessous. Il s'agit d'une classe de calculs mathématiques ne contenant qu'une méthode calculant la factorielle d'un entier.

```
public static class MathUtils
{
    public static int Factorial(int n)
    {
        if (n < 0) throw new ArgumentException("n must be non-negative");
        return n <= 1 ? 1 : n * Factorial(n - 1);
    }
}
```

- « **ProjectFile.csproj** » contenant le code donné ci-dessous. Il s'agit d'un fichier indispensable pour toute application .NET indiquant certains paramètres du projet.

  Dans cet exemple, il précise le type du projet (*Library donc non exécutable et ne nécessite pas de Main()*) et la version du Framework .NET à utiliser (*ici la version 8*).

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Library</OutputType>
    <TargetFramework>net8.0</TargetFramework>
  </PropertyGroup>
</Project>
```

3. Encore, dans le dossier « **TP3-DevOps** », ajoutez un sous-répertoire **« dotnet-app.Tests** », puis créez dedans les deux fichiers suivants :

   - Un fichier nommé **MathUtilsTest.cs** contenant le code donné ci-dessous. Il s'agit d'une classe de tests associée à celle du calcul de la factorielle précédente.

```
using Microsoft.VisualStudio.TestTools.UnitTesting;

[TestClass]
public class MathUtilsTests
{
    [TestMethod]
    public void TestFactorial5()
    {
        Assert.AreEqual(120, MathUtils.Factorial(5));
    }
}
```

   - Un fichier nommé **ProjectFile-Tests.csproj** contenant le code donné ci-dessous.

     Dans cet exemple, il précise la version .NET à utiliser ainsi que les dépendances requises en termes du Framework MSTest (*les packages et leurs versions*) et du projet à tester (*dernière ligne*).

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.9.0" />
    <PackageReference Include="MSTest.TestAdapter" Version="3.1.1" />
    <PackageReference Include="MSTest.TestFramework" Version="3.1.1" />
    <ProjectReference Include="..\dotnet-app\ProjectFile.csproj" />
  </ItemGroup>

</Project>
```

4. Lancez un navigateur Web, connectez-vous à votre compte GitHub et créez un dépôt public nommé `tp3-ci-tests` ayant une description de votre choix, par exemple « *CI pour tests .NET* ».

   NB : N'ajoutez ni le fichiers README, ni le fichier .gitignore, ni la licence.

5. Faites le nécessaire pour :
   - Rendre le dossier « **TP3-DevOps** » un dossier git.
   - Vérifier que vous êtes en train de travailler sur la branche ***main*** (pas ***master***).
   - Vérifier vos données de connexion à votre compte GitHub (***user.name*** et ***user.email***).
   - Indexer le contenu de votre dossier.
   - Créer le premier commit.
   - Associer le dépôt distant créé précédemment à votre dépôt local (***git remote add***).
   - Faire un push vers votre dépôt distant.
   - Vérifier que le push a bien réussi à transférer le contenu local vers votre dépôt GitHub.

B. Configuration du Workflow GitHub Actions

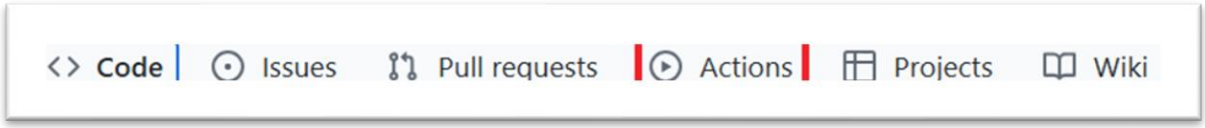
Pour l'automatisation des processus (de tests dans notre cas), **GitHub Actions** se base sur **des fichiers de workflow**.

Un fichier de workflow décrit un scénario automatisé que GitHub exécute à chaque **push** ou **pull request**.

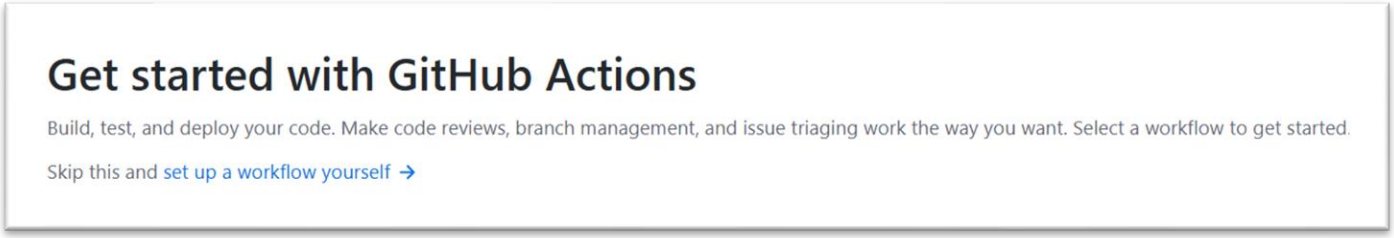
Il s'agit d'un fichier **YAML** qui définit un processus automatisé, comme la compilation, les tests ou le déploiement. Il spécifie les événements qui déclenchent le workflow (ex : push, pull request), les tâches (jobs) à exécuter et leurs environnements.

Il doit être placé dans un répertoire nommé « `.github/workflows` » du dépôt concerné, et son extension doit être `.yml` ou `.yaml`.

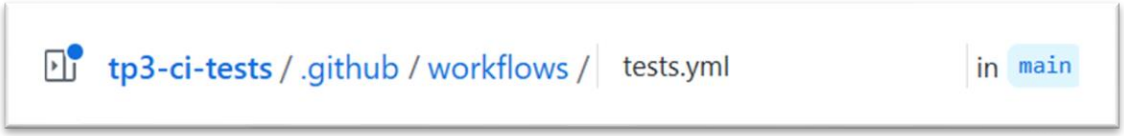
1. Dans **GitHub**, pour accéder à **GitHub Actions**, il suffit de cliquer sur l'onglet « **Actions** ».



   Puis, cliquez sur le lien « *set up a workflow yourself* » (en faut de la page affichée).



2. Copiez le code donné ci-dessous et collez-le dans l'éditeur qui a été ouvert sur GitHub Actions. Il s'agit du code du fichier workflow que vous nommerez **tests.yml** (à la place du nom par défaut « main.yml » affiché juste au-dessus de l'éditeur).



NB : Vous remarquerez déjà, que ce fichier sera créé dans le dossier « .github/workflows » sous votre dépôt.

```
name: Run Tests

on:
  push:
    branches: [ main ]

jobs:
  test-dotnet:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '8.0.x'

      - name: Restore dependencies
        run: |
          dotnet restore dotnet-app/ProjectFile.csproj
          dotnet restore dotnet-app.Tests/ProjectFile-Tests.csproj

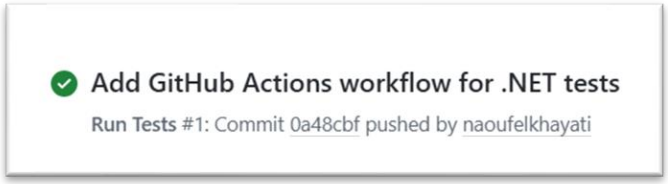
      - name: Run .NET tests
        run: dotnet test dotnet-app.Tests/ProjectFile-Tests.csproj
```

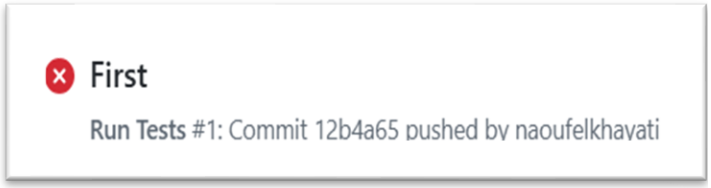
NB :

- ***Les indentations au niveau de ce fichier sont à respecter obligatoirement.***
- ***Ce fichier décrit un workflow nommé « Run Tests » qui s'exécutera automatiquement lors de chaque commande « push » exécutée sur la branche « main » ou chaque commit fait sur le dépôt distant.***
- ***Il crée une machine virtuelle Ubuntu (temporaire), dans laquelle il installe l'environnement .NET 8.***
  - ***GitHub Actions recrée un environnement vierge à chaque exécution (chaque job tourne dans une MV temporaire).***
- ***Il récupère les dépendances nécessaires à travers des fichiers .csproj.***
- ***Il exécute le test qu'il trouve dans le projet « dotnet-app.Tests ».***

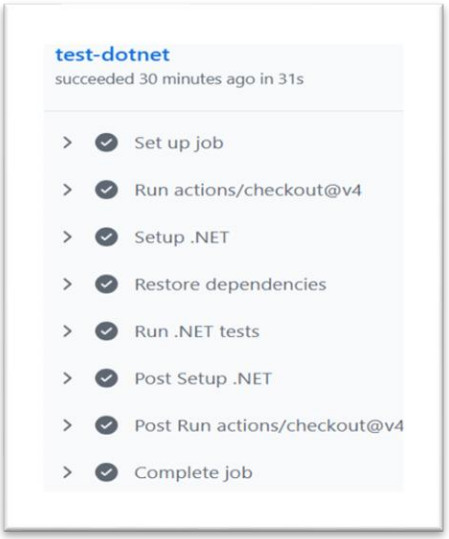
3. Cliquez ensuite sur le bouton vert en haut à droite « ***Commit changes…*** ».
   Dans la boite de dialogue qui s'affiche, vous pouvez donner votre propre message du commit ou garder celui suggéré par ***Copilot GitHub***. Enfin, cliquez sur « ***Commit changes*** ».

4. A la suite du nouveau commit, le test sera exécuté automatiquement et vous verrez un affichage indiquant son succès (figure de gauche) ou son échec (figure de droite).
   Normalement, vous devez voir un cas de succès du test.





5. Cliquez sur le résultat du test, puis sur le nom du job (**test-dotnet**). Ici vous verrez la liste des étapes suivies pour exécuter ce test (*Setup .NET, Restore dependencies, Run .NET tests, etc.*) comme le montre la figure ci-après.



6. Explorez ces différentes étapes en cliquant sur les flèches au début de chaque ligne et analysez les logs affichés.

   NB : Vous remarquerez que dans le log associé à l'étape d'installation de .NET indique que ce dernier est déjà installé. Ceci est dû au fait que GitHub met en cache automatiquement certains outils populaires comme .NET, Node.js et autres.

7. Maintenant que le contenu du dépôt distant a été modifié, vous ne pourrez plus faire des push faute de synchronisation des deux dépôts (local et distant). Il faut alors faire un « **git pull** » pour récupérer le fichier du workflow localement.
   NB : Une fois qu'il est sur votre disque, il doit rester suivi par Git.

8. Au niveau du log de l'étape d'exécution du test, vous remarquerez l'absence de toute information signalant que le test a vraiment bien réussi.
   Ainsi, pour améliorer l'affichage de cette étape, ajoutez l'option donnée ci-dessous à la dernière ligne du fichier de workflow qui vient d'être inséré dans votre dépôt local :

   ```
   --logger "console;verbosity=detailed"
   ```

   Ça devrait devenir ainsi :

   ```
   run: dotnet test dotnet-app.Tests/ProjectFile-Tests.csproj --logger "console;verbosity=detailed"
   ```

9. Faites le nécessaire pour pousser le nouveau code vers le dépôt distant.
   NB : Vous verrez que le test s'est déclenché automatiquement après le push et que le log de son exécution affiche bien un message signalant que TestFactorial5 a bien réussi (voir figure ci-après).



10. Ajoutez cette méthode à votre classe de tests locale (pour tester la factorielle de 0).

```
[TestMethod]
    public void TestFactorial0()
    {
        Assert.AreEqual(0, MathUtils.Factorial(0));
    }
```

11. Poussez vers le dépôt distant et vérifier le résultat du test.
    Vous remarquerez que le résultat indique le succès d'un test et l'échec d'un autre.
    En fait, le problème vient de notre méthode de test et non du code source.

12. Rectifiez cette dernière méthode afin que les deux tests passent avec succès.

C. Génération du rapport des tests

L'objectif maintenant est d'afficher les résultats de tests sous forme de rapports exploitables dans les pipelines GitHub Actions (fichiers TRX), pour une meilleure lisibilité et traçabilité.

1. Commencez par ajouter ceci à la fin de la ligne ***dotnet test*** du fichier YAML.

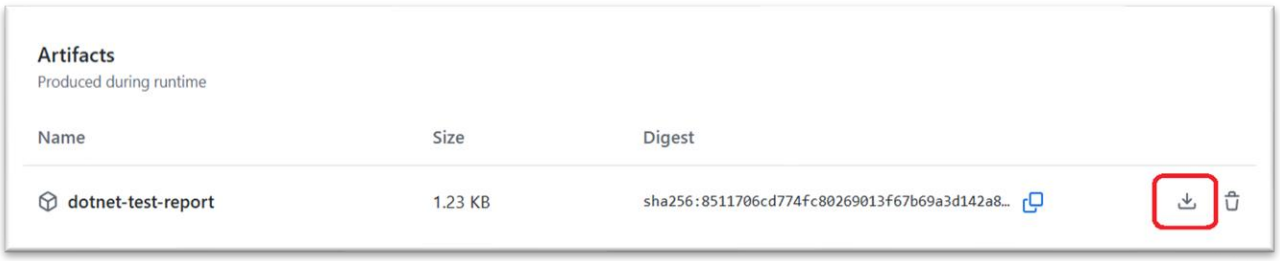
```
--logger "trx;LogFileName=TestResults.trx"
```

Cette option produit un fichier de log standard .trx (au format Visual Studio), que vous pourrez ultérieurement télécharger comme artefact GitHub Actions et consulter localement avec VS (Code) en ajoutant ce code à la fin du fichier YAML :

```
- name: Upload test report
  uses: actions/upload-artifact@v4
  with:
    name: dotnet-test-report
    path: dotnet-app.Tests/TestResults/TestResults.trx
```

2. Téléchargez le fichier .trx généré de l'une des méthodes suivantes :

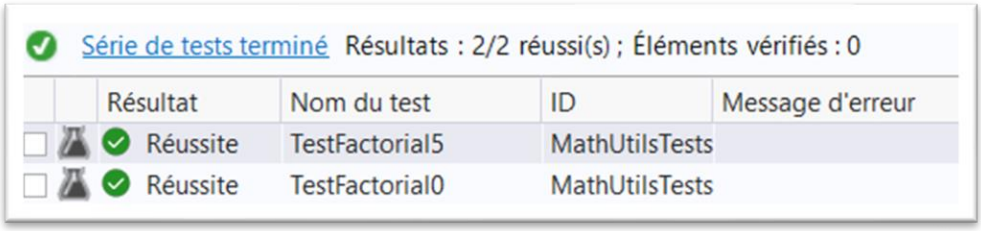
   - À partir du lien fourni dans la section « Artifacts » de la dernière exécution du test (voir figure ci-dessous).



   - À partir du lien fourni dans la partie affichant la liste des étapes du test et plus exactement de l'étape « ***Upload test report*** » (voir figure ci-dessous).



3. Décompressez le fichier .zip téléchargé et double-cliquez sur le .trx pour le visualiser.
   Il s'ouvrira sur Visual Studio et donnera un affichage comme suit :



D. Ajout d'un projet Python

1. Dans le dossier « **TP3-DevOps** », créez un nouveau sous-répertoire nommé « **python-app** ». Dans ce dernier, ajoutez les deux fichiers suivants :

   - « **math_utils.py** » contenant le code de la fonction ***factorial()*** donné ci-dessous

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    return 1 if n <= 1 else n * factorial(n - 1)
```

   - « **test_math_utils.py** » contenant le code de la classe de test suivant :

```
import unittest
from math_utils import factorial

class TestFactorial(unittest.TestCase):
    def test_factorial_5(self):
        self.assertEqual(factorial(5), 120)

if __name__ == '__main__':
    unittest.main()
```

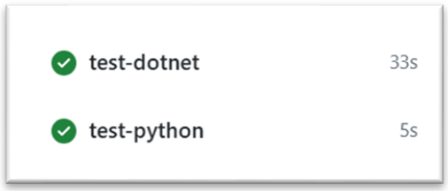
2. Ajoutez le job suivant à la fin du fichier YAML (***tests.yml***) :

```
test-python:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Run Python Tests
        run: |
          cd python-app
          python -m unittest
```

   Comme pour l'exemple .NET, ce job (***test-python***) a pour rôle de préparer une MV Ubuntu sur laquelle il y aura installation de l'environnement Python pour lancer le test grâce à la commande `python -m unitest`.

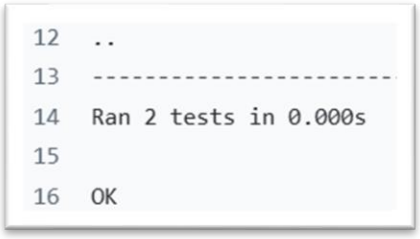
3. Poussez le tout vers le dépôt distant et regardez ce qui se passe pour ce nouveau job ***test-python***.

   Vous devez voir deux jobs (***test-dotnet*** et ***test-python***) dont les tests ont réussi.



4. Ajoutez une deuxième fonction de test au fichier ***test_math_utils.py*** (par exemple : `def test_factorial_0(self)`) pour le test du cas du zéro. Puis, poussez de nouveau vers GitHub et consultez le résultat de ***test-python***.

   Il devrait être comme suit :



   NB : Chacun des 2 points de la ligne 12 de la figure ci-dessus indique le succès du test python associé. En cas d'échec, vous verrez la lettre 'F' au lieu du point (`.F` ou `F.` ou `FF`).

5. Afin de pouvoir générer un rapport XML pour les tests python, vous allez procéder comme suit :

   - Installez le package de reporting à l'aide de cette commande (*sur une ligne de commande*) :

   ```
   pip install unittest-xml-reporting
   ```

   - Modifiez la classe de test pour devenir comme suit :

```
import unittest
import xmlrunner
from math_utils import factorial

class TestFactorial(unittest.TestCase):

    def test_factorial_5(self):
        self.assertEqual(factorial(5), 120)

    def test_factorial_0(self):
        self.assertEqual(factorial(0), 1)

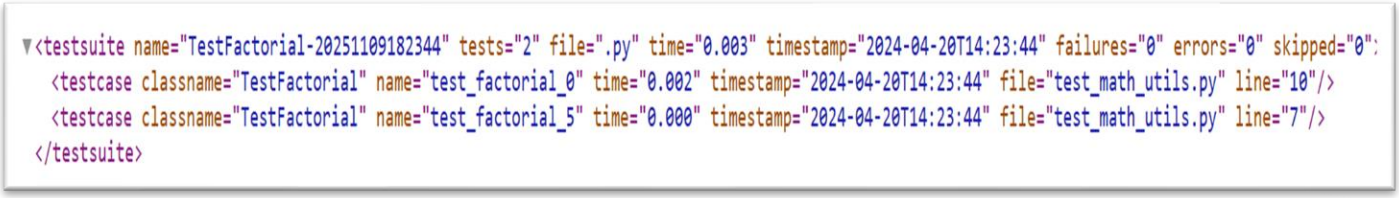
if __name__ == '__main__':
    unittest.main(testRunner=xmlrunner.XMLTestRunner(output='test-
reports'))
```

   - Dans le code du workflow YAML, remplacez la dernière section 'Run Python Tests) par ce code :

```
- name: Run Python Tests and Generate XML Report
  run: |
    cd python-app
    pip install unittest-xml-reporting
    python test_math_utils.py
- name: Upload Python Test Report
  uses: actions/upload-artifact@v4
  with:
    name: python-test-report
    path: python-app/test-reports
```

6. Comme pour l'exemple .NET, téléchargez ce rapport XML (***partie Artifacts***) et visualisez-le.

   Normalement, vous verrez un code similaire à celui-ci :



Partie 2 : Tests Automatisés avec Jenkins

Avant de commencer, il faut bien vérifier que vous avez Java 17 ou Java 21 installé sur votre machine car Jenkins n'accepte que l'une de ces deux versions. Ainsi, si vous avez une autre version, commencez par l'installation de Java 17 ou Java 21.

A. Installation de Jenkins

Etape 1 : Rendez-vous à l'URL https://www.jenkins.io/download/ afin de télécharger la version LTS de Jenkins pour Windows (*dans la liste de gauche*).

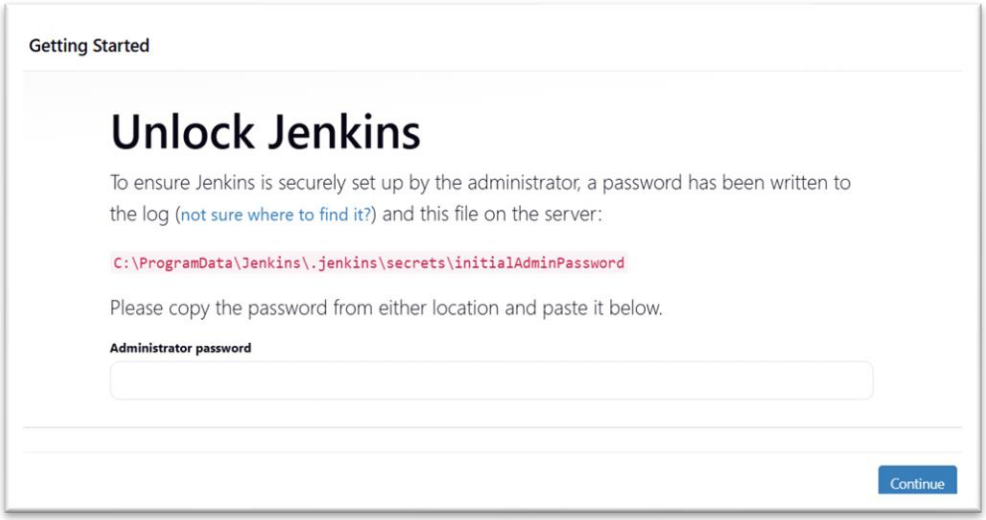
Etape 2 : Démarrez l'installation de Jenkins et suivez toutes les instructions.

- *Login Type : Run Service as LocalSystem (même si ce n'est pas recommandé)*
- *Vérifiez que le port 8080 est disponible (Test Port), sinon attribuez un autre port.*
- *Donnez le chemin vers votre Java (par exemple : C:\Program Files\Java\jdk-21\).*

Etape 3 : Une fois l'installation terminée, vérifiez que le serveur Jenkins est en cours d'exécution.

Ce serveur devrait maintenant être en cours d'exécution et est accessible via l'url **http://localhost:8080** à taper directement sur le navigateur.

Vous verrez un message pour débloquer Jenkins (*voir figure suivante*).



Etape 4 : Récupérez le mot de passe admin à partir du fichier *C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword*.

Ce mot de passe doit être copié puis collé dans la zone adéquate sur le navigateur afin de pouvoir terminer la configuration initiale du serveur Jenkins.

NB : Suivez à la lettre les indications de l'étape 5.

Etape 5 : Terminez la configuration initiale du serveur Jenkins.

NB : le nom d'utilisateur est « admin ».

- Installez les plugins Jenkins recommandés en cliquant sur « ***Install suggested plugins*** » et attendez que Jenkins termine cette tâche.
- Ignorez la création d'un nouvel utilisateur administrateur en cliquant sur « ***Skip and continue as admin*** » en bas.
- Ignorez la création d'une configuration d'instance en cliquant directement sur « ***Save and Finish*** ».
- Dans la fenêtre suivante, cliquez sur « **Start using Jenkins** ». Vous devriez maintenant être sur le tableau de bord principal local de Jenkins.

B. Configuration du Job Jenkins

L'unité fondamentale de Jenkins est dite **Job** (également connue sous le nom de projet). Vous pouvez créer des Jobs qui effectuent diverses tâches, notamment pour :

- Récupérer du code à partir d'un référentiel tel que GitHub.
- Générer une application à l'aide d'un script ou d'un outil de construction / de génération.
- Emballer une application et l'exécuter sur un serveur.

Dans cette section, vous allez créer un job Jenkins simple qui récupère la dernière version de votre application à partir de GitHub et exécute les tests déjà développés.

1. Cliquez sur « **Create a job** », attribuez-lui le nom « ***ci-tests-jenkins*** », choisissez le type « **Pipeline** » puis cliquez sur « **OK** ».
2. Dans la configuration, donnez une description quelconque à votre pipeline (par exemple, *Pipeline pour mes tests .NET et Python*), puis tout en bas, dans la section « **Pipeline** », choisissez à partir de la liste déroulante « ***Pipeline script from SCM*** ».

   Sous **SCM**, choisissez ***Git***, puis :

   a. Dans le champ URL du référentiel (*Repository URL*), ajoutez le lien de votre référentiel GitHub.

      `https://github.com/`***`github-nom_utilisateur`***`/tp3-ci-tests.git`

   b. Pour **Credentials** cliquez sur le bouton **Add** et choisissez **Jenkins**.

   c. Dans la boîte de dialogue **Add Credentials**, renseignez votre nom d'utilisateur GitHub et votre **jeton** (*Personal Access Token*), puis cliquez sur **Add**.

      NB : Ci-dessous les étapes de création du jeton.

- *Pour récupérer le token GitHub, suivez les étapes suivantes :*
  - *Dans le coin supérieur droit de n'importe quelle page sur GitHub, cliquez sur votre photo de profil, puis sur « **Settings** ». Dans la barre latérale gauche, tout en bas, cliquez sur « **Developer settings** ». Puis, encore dans la barre latérale gauche, sous « **Personal access tokens** », cliquez sur « **Fine-grained tokens** ». Cliquez ensuite sur le bouton « **Generate new token** ».*
  - *Saisissez un nom pour le jeton et sélectionnez un délai d'expiration pour ce jeton.*
  - *Sous « **Repository access** », sélectionnez la 3ème option, puis dans la liste déroulante qui apparait, choisissez le repo « **my-simple-web-app** » (Le jeton est donc autorisé à accéder à ce repo).*
  - *Sous « **Permissions** », sélectionnez les autorisations à accorder au jeton. Les deux permissions suivantes suffisent pour notre cas :*
    - *Contents: Read-only (pour que Jenkins puisse cloner votre code).*
    - *Metadata: Read-only (Jenkins utilise cette info pour identifier le dépôt).*
  - *Enfin, cliquez sur « **Generate token** ».*

d. Dans la liste déroulante des **Credentials** où il est actuellement indiqué **None**, choisissez les informations d'identification que vous venez de configurer (*Généralement il porte le nom d'utilisateur de votre compte GitHub, suivi de « /******* »*).

e. Remplacez la branche « ***master*** » par « ***main*** ».

Après avoir ajouté l'URL et les informations d'identification correctes, Jenkins teste l'accès au référentiel. Si un message d'erreur s'affiche, vérifiez votre URL et vos informations d'identification. Vous devrez les ajouter à nouveau car il n'y a aucun moyen à ce stade de supprimer ceux que vous avez entrés précédemment.

f. Cliquez sur le bouton « **Save** ».

A ce stade, votre pipeline est déjà prêt, mais non encore fonctionnel car il lui manque le **Jenkinsfile** que nous devrons ajouter au dépôt GitHub.

3. Ajoutez un fichier **Jenkinsfile** à la racine de votre projet local (dossier **TP3-DevOps**) et copiez dedans le code ci-dessous :

NB : Remplacez l'URL du répo GitHub par le vôtre.

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/……..git'
            }
        }

        stage('Run .NET tests') {
            steps {
                dir('dotnet-app') {
                    bat 'dotnet test'
                }
            }
        }

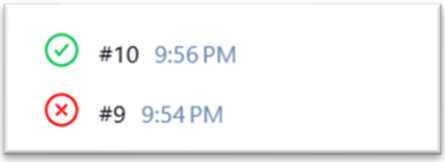
        stage('Run Python tests') {
            steps {
                dir('python-app') {
                    bat 'python -m unittest'
                }
            }
        }
    }

    post {
        always {
            echo 'Pipeline finished !!!'
        }
    }
}
```

Ce script décrit le pipeline Jenkins composé de deux tâches principales à savoir le lancement des tests .NET et puis ceux de Python.

4. Revenez à Jenkins et cliquez sur « ***Build Now*** » pour lancer le job créé précédemment.

   Si tout s'est bien passé, vous verrez dans la section ***Builds*** de gauche le lancement numéro 1 précédé d'une icône verte de succès, sinon il sera précédé d'une icône rouge d'échec (*voir exemple ci-dessous*).



   Pour voir les détails de l'exécution du job, cliquez sur le lien correspondant, puis cliquez sur **Console Output** (*à gauche*).

BON TRAVAIL